BEAML

Language Engineering Platform — Specification v0.3

Architecture baseline | 05 November 2026 | Design proposal, not an implemented system

Scope: a statically typed LLM-oriented language targeting the BEAM, with a Rust-implemented compiler, a unified semantic database, language server, property testing, domain contracts, architecture rules, and privacy-preserving hyperobservability.

Executive decisions

• Rust compiler and CLI; generated Erlang/Elixir-compatible modules; Elixir/OTP runtime.

• Lossless syntax tree, typed HIR, typed MIR, stable symbol IDs and incremental semantic queries.

• LSP, formatter, linter, testing and architecture checks reuse compiler semantics.

• Soundness boundaries: untrusted model responses must be validated; external Elixir calls are explicit trusted/checked boundaries.

• All numeric targets below are proposed acceptance thresholds, not measured results.

1. Product boundaries and non-goals

BEAML specializes in type-safe model interaction and workflow orchestration. It is not a replacement for Erlang, Elixir or Gleam. The domain/architecture language extensions are optional modules, not mandatory boilerplate. Avoid claims that a compiler can prove arbitrary business correctness, LLM truthfulness, or behavior of foreign code.

2. Language grammar and surface design

```
program     = { import | type_decl | trait_decl | impl_decl |
                client_decl | function_decl | property_decl |
                context_decl | architecture_decl } ;
type_decl   = ("type" | "newtype" | "class") IDENT type_params?
              ("=" type_expr | "{" field* "}") refinement? ;
type_params = "<" type_param ("," type_param)* ">" ;
type_param  = IDENT | "const" IDENT ":" type_expr ;
function_decl = "function" IDENT type_params? "(" params? ")"
                "->" type_expr effects? block ;
effects     = "!" ("pure" | "{" IDENT ("," IDENT)* "}") ;
property_decl = "property" STRING "forall" bindings block ;
architecture_decl = "architecture" IDENT "{" rule* "}" ;
```

Expression grammar is Pratt-parsed; templates have their own interpolated expression parser and precise source spans. The complete lexical and expression grammar remains a v0.4 deliverable; this excerpt is intentionally not advertised as a complete parser specification.

```
newtype InvoiceId = UUID
class Invoice deriving Schema, Validate, Arbitrary {
  id: InvoiceId
  amount: Decimal where self >= 0
  status: Draft | Approved | Paid
}
function Extract<T: Schema + Validate>(text: String) -> T
```

```
  ! {llm, network} {
  client Primary
  prompt """Extract {{ ctx.schema<T> }} from {{ text }}"""
}
```

3. Type-system contract

• Core: primitives, nominal newtypes, structural records, tagged ADTs, generics, exhaustive pattern matching, type inference for local expressions.

• Traits: coherent global instance resolution; no overlapping impls in v1; associated types in phase 2.

• Refinements: constant and interval proofs first; undecidable or unsupported predicates require explicit checked conversions.

• Effects: explicit sets and effect-subset checking, with pure functions unable to call effectful functions.

• Phantom types and typestate: Pending/Streaming/Validated/Complete states; affine ownership of stream handles later.

• Const generics: restricted natural-number expressions; higher-kinded types, GADTs and type families are later extensions.

• Schema reification: generic model outputs require a Schema witness, either monomorphized or passed as a typed dictionary.

Trust-boundary invariant

```
Untrusted<Json> --decode--> Parsed<T>
Parsed<T> --validate--> Result<Validated<T>, ValidationError>
Validated<T> --check refinement--> Result<Refined<T,P>, Error>
```

Validated values certify conformance to a declared schema, not factual truth. Runtime validation may fail; provider/network errors are represented in the generated API. Optional fields must distinguish missing from explicit null when the schema requires that distinction.

4. Compiler architecture and correctness

```
.beaml -> lexer -> lossless CST -> AST -> resolved HIR
      -> type/trait/effect/refinement checking -> typed MIR
      -> codegen (Elixir/Erlang source + validators + metadata)
      -> BEAM compiler -> .beam
```

Compiler implemented in Rust as a standalone executable invoked by Mix. Stable semantic IDs and source maps survive lowering. Typed MIR stores control flow, effects, validation obligations and instrumentation points. The first backend emits Elixir/Erlang-compatible source; direct BEAM bytecode is out of scope.

Incremental query graph

```
parse(file) -> lower(file) -> resolve(module)
type_of(def_id) -> effects_of(def_id)
mir_of(def_id) -> emit(module)
references(symbol_id) -> rename_edits(symbol_id)
```

Queries are deterministic with tracked dependencies and cycle diagnostics. Cache keys include compiler version, feature flags, schema capability descriptors and imported interface fingerprints. A warm-cache change to a prompt literal should not invalidate unrelated type queries.

Compiler correctness obligations

• Well-typed safe programs do not get stuck on language-level type mismatches (subject to explicit foreign and runtime effects).

• Every lowering pass preserves the observable semantics of the supported core language.

• Error recovery produces useful diagnostics without silently accepting ill-typed programs.

• Compile-fail suites, MIR golden tests, differential tests, fuzzing and property tests are release gates.

5. Language server and developer experience

beaml-lsp uses the same semantic database as the compiler. It maintains overlays for unsaved buffers and reparses only affected syntax regions where possible. It supports incomplete and malformed code without crashing.

Capability	Acceptance behavior
Completion	Type-aware fields, traits, functions and prompt interpolation
Navigation	Definition, references, implementation, call hierarchy
Refactor	Rename with cross-file edits and collision checks
Diagnostics	Syntax, type, trait, effect, refinement, architecture
Editor support	Hover, semantic tokens, inlay hints, code actions, formatting

LSP compatibility is verified against the published protocol version chosen for release. Editor plugins must not duplicate semantic analysis. All edits are versioned and rejected when the client document version is stale.

6. Property-based testing as a first-class language facility

```
property "totals are nonnegative" forall invoice: Invoice {
  assert invoice.amount >= 0
}
property "roundtrip" forall invoice: Invoice {
  assert decode<Invoice>(encode(invoice)) == Ok(invoice)
}
```

Deriving Arbitrary builds generators from types. Refinements use constructive generation or constraint solving when available; failure to generate a valid case is reported as generator exhaustion, not a passing property. Shrinking preserves the declared type and refinements at every accepted step.

State-machine and model-based testing

```
model InvoiceLifecycle {
  state Draft | Approved | Paid
  command approve: Draft -> Approved
  command pay: Approved -> Paid
  invariant "payment requires approval" {
    state == Paid => history.contains(approve)
  }
}
```

The runner generates command sequences, executes a reference model and system-under-test adapter, and shrinks failing traces while preserving preconditions. It records seed, compiler version, generator version and minimal counterexample. Stateful concurrency testing is a separate phase with explicit

scheduler instrumentation.

7. DDD: domain types, aggregates, commands, events

```
context Billing {
  newtype InvoiceId = UUID
  aggregate Invoice {
    id: InvoiceId
    status: InvoiceStatus
    invariant { total.amount >= 0 }
    command approve() requires status == Draft
      ensures status == Approved
    event InvoiceApproved { invoice_id: InvoiceId }
  }
}
```

DDD constructs are optional annotations with explicit semantics. Pure aggregate transitions can be statically analyzed; database atomicity, external side effects and distributed consistency require runtime and integration guarantees. Generated event schemas are versioned, with compatibility checks for breaking changes.

8. ArchUnit-style architecture constraints

```
architecture BillingRules {
  rule "domain is pure" {
    modules("billing.domain.**")
      .must_not_have_effects {network, database, llm}
  }
  rule "no dependency cycles" {
    packages("billing.**").must_be_acyclic()
  }
  rule "sensitive telemetry flow" {
    fields(annotated Sensitive)
      .must_not_flow_to(effect telemetry)
  }
}
```

Structural checks use the symbol/dependency graph; semantic checks use typed HIR/MIR and effect sets. Information-flow rules require conservative taint analysis and may be inconclusive at opaque foreign calls. Inconclusive results are surfaced, never presented as proven safe.

9. Contracts as Code: proof status is mandatory

```
contract InvoiceIntegrity {
  invariant invoice.total >= 0
  property "sum of items" {
    invoice.total == sum(invoice.items.amount)
  }
  architecture { domain.must_be_pure() }
  observe { violations: always; capture: metadata }
}
```

Status	Meaning	Release treatment
PROVEN	Discharged by sound supported static rules	Accept
ENFORCED	Runtime validator inserted at boundary	Accept with runtime tests
TESTED	Observed over generated/sample executions	No proof claim
UNKNOWN	Solver or analysis inconclusive	Explicit waiver or reject

Status	Meaning	Release treatment
VIOLATED	Counterexample or static violation	Reject

The compiler emits machine-readable contract manifests containing obligation ID, source span, checker version, status, evidence reference and runtime validation hook where relevant.

10. Hyperobservability and privacy

Instrumentation is compiler-generated with stable operation IDs, trace/span context, type/schema versions, effect metadata, validation-path identifiers and source maps. Logical workflow edges are distinguished from BEAM process links. Context propagation across process messages is explicit.

• Metrics: provider latency, time to first token, tokens, cost estimate, retries, validation failure rate, field readiness and queue time.

• Default telemetry captures metadata only; sensitive prompts, responses and field values are redacted or absent.

• Replay stores explicitly permitted fixtures and recorded provider responses; it is not deterministic replay of arbitrary BEAM scheduling.

• Telemetry exporters and logs are themselves treated as external effects; no sensitive field may be emitted without policy approval.

11. Performance and reliability acceptance targets

These are proposed thresholds for a reference Linux x86-64 CI runner with 8 dedicated vCPUs, 32 GiB RAM, local NVMe, warm OS filesystem cache, and pinned compiler/runtime versions. Benchmarks run at least 30 iterations; report p50, p95, maximum and confidence interval. Network/model latency is excluded from compiler and LSP benchmarks.

Metric	Target	Fixture / measurement
Full compile	p95 <= 2.0 s	10k LOC, 100 modules, warm disk, cold semantic cache
Incremental compile	p95 <= 200 ms	One function edit, 10k LOC
LSP completion	p95 <= 100 ms	10k LOC, warm query cache
LSP hover	p95 <= 50 ms	10k LOC, warm query cache
LSP definition	p95 <= 75 ms	10k LOC, warm query cache
LSP rename	p95 <= 500 ms	1,000 references, 10k LOC
Architecture check	p95 <= 1 s	100k LOC, warm semantic cache
Property runner	>= 10k cases/s	Pure scalar property, 1 thread
Compiler RSS	<= 1.5 GiB	100k LOC, clean build
Telemetry overhead	<= 5% CPU	Synthetic runtime workload, metadata mode
Fuzz robustness	0 crashes	>= 10M malformed inputs across parsers

A cache-hit ratio of >=95% is an optimization target only for the controlled single-function-edit fixture; cache hit ratios are workload-dependent. Property throughput must be reported separately for scalar, nested/refined and stateful generators. Fuzzing zero crashes is evidence, not a mathematical proof of safety.

12. Test strategy and release gates

• Every syntax feature: parser fixtures, malformed-input recovery and formatter idempotence tests.

• Every type feature: positive/negative typing cases, error codes and property tests for substitution/inference.

• Every optimization: MIR semantic differential tests and reproducible codegen.

• Every LSP feature: protocol integration tests, incomplete-source fixtures and stale-edit handling.

• Every contract checker: soundness assumptions, proof-status outputs, counterexamples and foreign-boundary tests.

• Every runtime provider: schema decoding, cancellation, timeout, retries and telemetry privacy tests.

• Release gate: no unresolved P0/P1 defects, all mandatory tests pass, no regression beyond budget without signed exception.

13. Repository architecture

```
crates/
  beaml_syntax/      beaml_hir/         beaml_types/
  beaml_traits/      beaml_effects/     beaml_mir/
  beaml_query/       beaml_codegen/     beaml_lsp/
  beaml_formatter/   beaml_property/    beaml_architecture/
  beaml_contracts/   beaml_cli/
beam/
  beaml_runtime/     beaml_testing/     beaml_telemetry/
editor/
  vscode/
benchmarks/  fixtures/  specifications/
```

Crate boundaries describe intended modules; initial implementation may combine small crates to avoid premature fragmentation. Shared semantics must not be duplicated in LSP, testing or architecture components.

14. Milestones and objective exit criteria

Phase	Deliverable	Exit criterion
M0	Formal core and spec	Typing/evaluation rules, 100 normative examples
M1	Parser + CST + diagnostics	1M parser fuzz cases, zero crashes
M2	Types, generics, ADTs	500 compile-pass/fail fixtures
M3	MIR + BEAM backend	End-to-end validated extraction demo
M4	Incremental + LSP	Hover/definition/rename, target latency tests
M5	Properties + shrinking	Seed replay, invariant-preserving shrink
M6	DDD + architecture rules	Static checks and model-based tests
M7	Observability + replay	Trace graph, redaction tests, fixture replay
M8	Hardening + release	Performance budgets and 10M fuzz cases

15. Open design decisions and risks

• Precise subtype relation for structural records and variance of generic parameters.

• Trait coherence and orphan rules across packages; associated-type normalization.

• Refinement predicate fragment and solver timeout/unknown policy.

• Streaming field states and whether partial values may be revised before completion.

• BEAM interoperability: opaque foreign effects, dynamic atom safety, serialization and process cancellation.

• Effect handler strategy: explicit handler dictionaries first, algebraic continuations later.

• Architecture taint analysis precision and waiver governance.

• Schema evolution and compatibility guarantees for domain events and generated APIs.

16. Normative language for future specifications

MUST denotes a release-blocking requirement. SHOULD denotes a recommended default with documented exceptions. MAY denotes optional functionality. No performance number is a claim of achieved implementation performance until benchmark evidence is attached. A language feature is not stable without documented syntax, typing, dynamic semantics, diagnostics, interoperability behavior and tests.

Version 0.3 is an architectural baseline. Next specification deliverable: formal typing judgments, operational semantics, trait-resolution rules, typed MIR invariants, exact query invalidation protocol and complete EBNF.

BEAML v0.4 — Phoenix LiveView & Type-Aware HEEx

Normative extension to Engineering Specification v0.3 | 05 November 2026 | Status: proposed design; no implementation or benchmark is claimed. The preceding v0.3 specification is retained unchanged. This appendix defines an incremental, compiler-driven UI contract layer rather than a new rendering language.

21. Architectural boundary and ownership

BEAML owns domain types, application commands, validation, effects, and optionally UI contracts. Elixir owns Phoenix LiveView callbacks, HEEx markup, components, routing, Ecto adapters, and OTP infrastructure. The boundary is a generated, versioned Elixir ABI with explicit decoding and validation. Direct construction of ordinary Elixir structs does not confer trusted BEAML type status.

```
BEAML domain + application contracts
        | schema / contract descriptors
        v
Generated Elixir API + Phoenix adapter
        |
Phoenix LiveView + HEEx templates
        ^
Type-aware HEEx analyzer <--> shared semantic query engine
```

22. Contract syntax: assigns, events, forms and components

Illustrative BEAML syntax, to be formalized in the grammar. UI contracts are optional and may be attached to a LiveView, component, or command. External data enters through explicit decoder functions.

```
ui InvoiceEditor {
  assigns { invoice: Invoice; form: Form<CreateInvoice.Input>; busy: Bool }
  event "validate"(payload: FormParams<CreateInvoice.Input>)
  event "save"(payload: FormParams<CreateInvoice.Input>)
  event "delete"(payload: { id: InvoiceId })
  component InvoiceCard {
    prop invoice: Invoice
    prop compact: Bool = false
    slot actions: Slot<InvoiceId> optional
  }
}
```

Semantics: event names are literal discriminants; payloads are decoded from untrusted string-keyed maps; assigns are checked at render entry and across recognized assign transformations. Conditional branches, dynamic keys, and arbitrary Elixir metaprogramming require explicit fallback diagnostics rather than false guarantees.

23. Generated Elixir / Phoenix adapter

```
alias MyApp.BEAML.InvoiceEditor, as: Contract

def mount(_params, _session, socket) do
  {:ok, Contract.assign(socket, invoice: invoice,
    form: Contract.form(), busy: false)}
end

def handle_event("save", raw, socket) do
  with {:ok, input} <- Contract.decode_event(:save, raw),
       {:ok, invoice} <- CreateInvoice.execute(input) do
    {:noreply, Contract.assign(socket, invoice: invoice)}
  else
    {:error, error} -> {:noreply, Contract.put_error(socket, error)}
  end
end
```

The generated ABI must expose stable typed descriptors, decode_event/2, form/0, change/2, assign/2 and field references. Event decoding does not assume that browser payloads are valid. Return values remain explicit {:ok, value} / {:error, error} at the Elixir boundary.

24. Type-aware HEEx analysis

A dedicated analyzer uses the official Phoenix/HEEx parser and available Elixir semantic information, mapped to BEAML HIR symbol IDs and source spans. It must not rely on regex over template text. The initial analysis is sound only for the supported, statically analyzable HEEx subset.

```
<.input field={Contract.field(@form, :amount)} />
<.invoice_card invoice={@invoice} compact={true}>
  <:actions :let={id}><.button phx-click="delete"
    phx-value-id={id}>Delete</.button></:actions>
</.invoice_card>
```

Checks: (a) @assign presence and declared type, (b) field names and refinement metadata, (c) component prop and slot shape, (d) literal phx-event name and payload contract, (e) missing event handlers where statically knowable, (f) unsafe dynamic boundaries flagged as unknown. In particular, phx-value-* values arrive as strings and require decoding; an InvoiceId is never inferred from a raw browser string.

25. Diagnostic specification and LSP capabilities

```
error[BEAML-UI-001]: unknown field `ammount`
 --> lib/my_app_web/live/invoice_live.html.heex:17:32
  | field={Contract.field(@form, :ammount)}
  |                              ^^^^^^^
  = help: did you mean `amount`?
  = fix: replace `ammount` with `amount`
```

Code	Condition	Required action
UI-001	Unknown field / assign	Precise span + rename fix
UI-002	Prop or slot type mismatch	Expected/actual type + source
UI-003	Unknown event name	Suggest declared events
UI-004	Event payload mismatch	Show decoder requirement
UI-005	Unsupported dynamic expression	Report analysis limitation, not success
UI-006	Missing required assign	Show required render contract

LSP v1: completion, hover, go-to-definition, references, rename, semantic tokens, code actions, and diagnostics in .beaml, .heex, and supported ~H sigils. A single semantic query database supplies BEAML facts; HEEx and Elixir parsing remain separate frontends. Rename must preserve source mappings and refuse unsafe dynamic transformations.

26. Forms and validation semantics

Three states are distinct: RawFormParams -> ParsedInput -> Validated. Partial form editing returns field-level diagnostics without claiming a fully valid aggregate. Server-side validation is authoritative. Client-side constraints generated from refinements are convenience hints only. Optional, missing, empty-string and null values must have separately specified semantics.

```
CreateInvoice.form()
CreateInvoice.change(form, raw_params) # partial, field errors
CreateInvoice.decode(raw_params)       # Result<Validated<Input>, Error>
CreateInvoice.execute(validated)       # Result<Invoice, DomainError>
```

27. Architecture and security contracts

Architecture rules must prevent web modules from being imported by pure domain modules, and prohibit sensitive data from telemetry sinks in statically tracked flows. Dynamic Elixir calls, macros, NIFs, reflection and untracked libraries are explicit trust boundaries. Every claim must be classified PROVED, RUNTIME_ENFORCED, TESTED or UNKNOWN.

```
architecture WebBoundary {
  rule domain_is_pure: modules("domain.**").effects == {}
  rule no_web_dependency: modules("domain.**")
    .must_not_depend_on("**Web.**")
  rule web_calls_application: modules("**Web.**")
    .may_call("application.**", "ui.**")
}
```

28. Property tests and integration tests

Generators derive from BEAML refinements where constructively possible; generated values and shrink steps must preserve the declared input type. Cross-language tests cover BEAML -> generated API -> LiveView event decoding -> application command -> rendered HEEx. Tests must include invalid/missing keys, nested forms, malicious event payloads, optional fields, union variants, slot mismatches, and re-render after errors.

```
property "valid form roundtrips" forall input: CreateInvoice.Input {
  let encoded = Form.encode(input)
  assert CreateInvoice.decode(encoded) == Ok(input)
}

property "invalid payload never executes command"
  forall raw: Json where !CreateInvoice.accepts(raw) {
  assert submit(raw).domain_effect_count == 0
}
```

29. Performance budgets and reproducible benchmarks

All values below are proposed acceptance targets, not measurements. Benchmark on a pinned machine (8 modern performance cores, 32 GiB RAM, local SSD), warm and cold caches reported separately, 30 runs, p50/p95/p99, with fixtures of 10k and 100k BEAML LOC plus 100 and 1,000 HEEx templates. Exclude editor/network latency; record compiler revision, OTP/Phoenix versions, and OS.

Metric	Proposed gate	Fixture / method
HEEx completion p95	<= 100 ms	1k templates; warm index
HEEx hover p95	<= 50 ms	1k templates; warm index
Incremental diagnostics p95	<= 200 ms	Single-template edit
Project-wide rename p95	<= 500 ms	1,000 static references
Architecture check p95	<= 1 s	100k LOC; warm cache
Semantic correctness	>= 99%	Curated labeled fixture suite
False-safe classification	0 known	Unsupported syntax -> UNKNOWN
Fuzz crash count	0 / 10M inputs	Parser + HEEx adapter corpus
Runtime validation bypass	0 in conformance suite	All generated public entry points

Performance gates are aspirational until measured. No benchmark can establish soundness by itself. Semantic correctness is measured on a versioned fixture corpus, with separate false-positive and false-negative counts.

30. Delivery phases and acceptance gates

Phase	Deliverable	Release gate
A	UI contract grammar + typed descriptors	Golden tests + grammar tests
B	Generated Phoenix form/event adapters	End-to-end validation tests
C	HEEx parser + semantic cross-references	Unknown boundaries reported
D	LSP diagnostics + completion + rename	Performance and fixture gates
E	Props, slots, assigns, effects analysis	Regression + architecture tests
F	Property-based UI contract testing	Reproducible seeds + shrinking

31. Non-goals and open decisions

Non-goals for v0.4: replacing HEEx, compiling arbitrary Elixir to BEAML, statically proving arbitrary LiveView state transitions, or treating generated client-side validation as a security boundary. Open decisions: Phoenix parser compatibility matrix; schema descriptor format; component attribute/slot typing conventions; support for dynamic components; cross-language type metadata transport; compile-time vs CI enforcement policy.

32. Normative definition of done

A BEAML Type-Aware HEEx feature is complete only when its grammar, typing rule, source mapping, generated ABI, diagnostics, negative tests, dynamic-boundary behavior, and benchmark fixture are documented. Unknown analyses must never be silently presented as proved. All generated entry points must validate untrusted browser input before domain execution.

BEAML v0.5 — Observatory & Information Flow Security

Normative proposed extension to v0.4 | 05 November 2026. All earlier chapters are retained. This appendix introduces compiler-generated observability and information-flow controls. No security proof, benchmark result, or implemented feature is claimed.

33. Security objectives and non-guarantees

The primary objective is to prevent unauthorized sensitive-value flows across compiler-controlled boundaries by default. A compiler alone cannot promise that secrets never leak: arbitrary Elixir, NIFs, compromised hosts, side channels, explicitly authorized recipients, and misconfigured infrastructure remain in the threat model. Therefore all assurances are scoped, testable, and tied to a declared trusted computing base (TCB).

34. Classification and typed labels

```
type Email = String @pii(email)
type AccessToken = Bytes @secret(credential)
type CustomerId = UUID @confidential(customer)

class Customer {
  id CustomerId
  email Email
  notes String @pii(free_text)
}

fn display_name(c: Customer) -> String @pii(free_text) {
  return "Customer: " + c.email
}
```

The label is attached to the value and propagates through string interpolation, concatenation, formatting, collections, records, variants, maps, errors and derived values. A bare String return annotation must not silently erase a sensitive label; the compiler rejects it or infers the sensitive result. Implicit conversions to less-restricted labels are forbidden.

35. Information-flow lattice and effect constraints

Use a label lattice with dimensions **confidentiality** (public, internal, confidential, secret), **data categories** (PII kinds, credentials, financial, health), and **allowed purposes/recipients**. Join combines the restrictions of all inputs. The core typing judgment is Gamma; pc |- e : T @ L ! E. The program-counter label pc tracks implicit flows through secret-dependent branches; pure value-label propagation alone is insufficient.

```
label(join(a,b)) = join(label(a), label(b))
if secret_condition { emit("yes") } else { emit("no") }
// Both outputs carry the implicit-flow restriction from pc.
// Sink check: can_flow(label(value), sink_policy, context) == true
```

Public error strings, branch-dependent logs, timing, response sizes, and cardinality can disclose information even without raw secret bytes. For v1, reject secret-dependent output branches to public sinks where analyzable; explicitly document that timing and other side channels are not comprehensively eliminated.

36. Sink policies and explicit release

```
sink telemetry.metadata accepts Public | ApprovedAggregate
sink browser.render accepts Public | UserScopedPII
sink llm.external accepts Public | ExplicitlyApprovedPII
sink http.external accepts Public | RecipientScopedData

fn masked_email(x: Email) -> String @public
  ! {declassify(reason: "display masked email")} {
  return privacy.mask_email(x)
}
```

Declassification is a privileged capability, not a cast or arbitrary annotation. Require a named policy, justification, principal/purpose, reviewer-controlled allowlist, audit event containing only metadata, and a generated callsite identifier. Prefer vetted operations (masking, irreversible tokenization, aggregation) over free-form declassification. A mask is not automatically anonymous; the declassification rule must specify its leakage budget and output guarantee.

37. Elixir / Phoenix trust boundaries

BEAML-generated public APIs use opaque handles or sealed validated values where possible. At Elixir interop boundaries, treat unknown functions as potentially leaking unless they are in a reviewed effect/taint manifest. Dynamic dispatch, macros, reflection, NIFs, and untracked dependencies are UNKNOWN or blocked in strict mode. Ordinary Elixir structs do not enforce secrecy. HEEx analysis checks rendering, assigns, component props, and event payloads; legitimate user-scoped PII display requires explicit authorization, not global declassification.

```
ui CustomerProfile {
  assigns { customer: Customer @visible_to(owner_or_admin) }
  render_policy { require principal.can_view(customer.id) }
}
// @customer.email rendered only after authorized view policy.
// No PII in phx-value-* attributes, URLs, debug inspect or flash.
```

38. Secret handling and cryptographic boundaries

Secrets are sourced from a secret manager or runtime environment, not embedded in .beaml source or compiler artifacts. Runtime representation should minimize copies, prevent routine inspect/serialization, and zeroize where feasible; BEAM garbage collection prevents a blanket memory-erasure guarantee. Encrypt at rest and in transit, use least-privilege access, rotate credentials, and disable crash-dump/core-dump secret exposure where operationally feasible. Redaction is defense in depth, not the primary control.

Observatory — operational specification

39. Capture architecture and data ownership

BEAML compiler instrumentation -> Phoenix/OTP telemetry adapter -> local redaction and policy enforcement -> OpenTelemetry Collector -> storage. Deployment modes: local, self-hosted, and optional managed. The default self-hosted reference deployment stores traces/events in ClickHouse, application/project metadata in PostgreSQL, and optional encrypted replay artifacts in S3-compatible storage. A small deployment may store traces in PostgreSQL initially. No remote exporter is required.

40. Telemetry event schema

```
ObservationEvent {
  event_id: UUID; trace_id: TraceId; span_id: SpanId
  parent_span_id: Option<SpanId>; process_id: OpaquePid
  semantic_id: DefinitionId; source_map_id: SourceMapId
  event_kind: EventKind; timestamp: Instant
  duration_ns: Option<Int>; outcome: Outcome
  attributes: Map<String, PublicTelemetryValue>
  schema_version: Int
}
```

Events carry stable compiler semantic IDs, source maps, trace context across OTP process boundaries, and sanitized public attributes. Payload capture is OFF by default. Secret/PII values must not be stringified, inspected, hashed for stable identifiers, or serialized into traces. Even metadata such as customer IDs, paths, exception messages and prompt templates may be sensitive.

41. Sampling, retention, replay and backpressure

Dataset	Default proposal	Safety / operation
Technical traces	14 days	Sampling; no payloads
Error traces	30 days	Policy-filtered; errors may contain PII
Aggregated metrics	90 days	Bounded label cardinality
Prompt/response bodies	OFF	Explicit consent and purpose required
Replay artifacts	OFF / opt-in 7 days	Encrypted; scoped access; delete support

Use bounded buffers, batching and loss counters. On collector outage, business operations must not be blocked by routine telemetry; discard low-priority observations rather than persisting sensitive data in unbounded local spools. Security-audit events requiring durable delivery use a separate, explicit policy and failure behavior. Provider replay is not equivalent to deterministic BEAM process replay.

42. Observatory product surfaces

Trace explorer (LiveView event -> BEAML command -> Ecto/LLM -> OTP process); semantic type and contract violation explorer; process and supervision topology; model latency/token/cost metrics; source-aware exception navigation; privacy-policy violation report; release-policy audit trail; sampling/retention administration. Dashboards must obey tenant isolation and role-based access; access to observability data is itself audited.

Security assurance, tests and delivery

43. Compiler passes and IR invariants

Introduce a security-label inference pass over typed HIR, control-flow-sensitive propagation over MIR, and a sink-policy verifier before codegen. IR lowering and optimizer passes must preserve labels and security effects. Generated BEAM/Elixir boundaries receive runtime guards where static guarantees end. Every permitted release is explicit in the IR; compiler output contains a machine-readable security manifest and source map.

```
parse -> resolve -> typecheck -> label inference -> MIR
  -> control-flow/implicit-flow analysis -> sink verifier
  -> instrumentation/redaction lowering -> BEAM codegen
  -> policy manifest + diagnostics + Observatory metadata
```

44. Mandatory negative tests and security gates

Gate	Target (proposed)	Validation
Unauthorized sink flows	0 accepted in suite	Golden compile-fail cases
Label preservation	100% tested IR passes	Property + metamorphic tests
Telemetry PII leaks	0 in canary corpus	End-to-end sink inspection
Fuzz stability	0 crashes / 10M cases	Parser + label engine + adapters
Interop policy	100% callsites classified	Manifest CI check
Secret logging	0 accepted in strict mode	Elixir/HEEx boundary tests

Test direct and implicit leaks; nested structs; errors; exceptions; inspect; format; interpolation; Phoenix HTML and phx-value attributes; PubSub; Ecto logs; HTTP clients; LLM prompts; Oban job arguments; crash dumps; dynamic interop. Property-test classification joins and label-preserving shrinking. Red-team CI injects unique canary secrets and searches all configured sinks; this is evidence, not a proof of impossibility.

45. Release policy and rollout

P0: annotations, label propagation, strict sink checks, generated redacted telemetry, explicit unknown interop diagnostics, minimal Observatory with OTLP + Postgres. **P1:** ClickHouse, process graphs, policy-aware HEEx analysis, declassification approvals, security manifests and CI gates. **P2:** advanced implicit-flow analysis, vetted masking/tokenization, optional encrypted replay, distributed lineage and stronger formal verification. Default for production: fail closed for unclassified sensitive egress, payload capture disabled, and no implicit declassification.

46. Definition of done

Each security rule needs syntax, formal flow rule, static/dynamic enforcement boundary, positive and negative tests, diagnostic code, interop behavior, threat-model limitation and operational runbook. Security status is reported as PROVED (within formal model), RUNTIME_ENFORCED, TESTED or UNKNOWN; no blanket no-leak claim is permitted.